

Topic 4: The Confusion Matrix and Classification Metrics

0.1 The Confusion Matrix

0.1.1 Why “How Accurate?” Is Not Enough

Once a classifier is trained, the obvious question is “how good is it?”. The tempting answer — the percentage of correct predictions — hides crucial detail. A medical test can be wrong in two very different ways: telling a healthy person they are sick (a false alarm) or telling a sick person they are healthy (a dangerous miss). These two mistakes have completely different costs, and a single accuracy number cannot distinguish them. The **confusion matrix** lays out every kind of right and wrong answer in one small table.

Definition: Confusion Matrix

For binary classification, the confusion matrix is a 2×2 table that counts how predictions line up with reality. Calling one class *positive* (usually the class we care about detecting, e.g. “sick”, “spam”, “fraud”) and the other *negative*:

- **True Positive (TP)**: actually positive, predicted positive. *Correct hit*.
- **True Negative (TN)**: actually negative, predicted negative. *Correct rejection*.
- **False Positive (FP)**: actually negative, predicted positive. *False alarm* (Type I error).
- **False Negative (FN)**: actually positive, predicted negative. *Miss* (Type II error).

A memory aid: the *second* word (Positive/Negative) is what the model *said*; the *first* word (True/False) says whether the model was right.

Table 1: The layout of a binary confusion matrix.

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

The green diagonal holds the correct predictions; everything off the diagonal is a mistake — and the two kinds of mistake sit in different cells, exactly the distinction accuracy erases.

Worked Example: A Disease-Screening Test on 1,000 Patients

A hospital screens 1,000 patients for a disease that 50 of them actually have. The model's results:

		Predicted	
		Sick	Healthy
Actual	Sick (50)	TP = 40	FN = 10
	Healthy (950)	FP = 30	TN = 920

Reading the story out of the numbers:

- 40 sick patients were correctly flagged (TP) — they can begin treatment.
- 10 sick patients were sent home as healthy (FN) — the most dangerous cell in this context.
- 30 healthy patients were wrongly flagged (FP) — they face stress and follow-up tests, but no missed disease.
- 920 healthy patients were correctly cleared (TN).

Accuracy is $(40 + 920)/1000 = 96\%$, which sounds excellent — yet the model missed $10/50 = 20\%$ of all sick patients. Keep this example in mind: the next section builds every metric from these same four numbers.

A caution about scikit-learn's layout. scikit-learn's `confusion_matrix` orders the matrix by label value, so row 0 / column 0 refer to class 0. Its layout is $\begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix}$ — TN in the top-left, TP in the bottom-right. Textbooks often draw it the other way around; always check which convention is in use before reading numbers. The companion notebook demonstrates this side by side.

Common Mistakes

- **Mixing up FP and FN.** Anchor yourself: False Negative = a real positive that the model *missed*.
- **Assuming scikit-learn's layout matches the textbook layout.** It usually does not (see above).
- **Forgetting to state which class is “positive”.** Every cell's meaning flips if the positive class flips.
- **Treating both error types as equally bad.** Their real-world costs are almost never equal.

Key Points

- The confusion matrix separates the two kinds of error (FP and FN) that accuracy blends together.
- TP/TN are correct; FP is a false alarm; FN is a miss.
- All classification metrics in the next section are built from these four counts.

0.2 Classification Metrics

All the metrics below are simple recipes using the four confusion-matrix counts (TP, TN, FP, FN). Throughout, we reuse the disease-screening example: TP = 40, FN = 10, FP = 30, TN = 920.

0.2.1 Accuracy: The Headline Number (Handle With Care)

Definition: Accuracy

The fraction of all predictions that were correct:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

For the screening example: $(40 + 920)/1000 = 0.96$.

The Accuracy Trap on Imbalanced Data

Imagine a lazier “model” that simply predicts *healthy* for everyone. It scores $950/1000 = 95\%$ accuracy while detecting **zero** sick patients. When one class vastly outnumbers the other (fraud detection, rare diseases, spam), accuracy rewards doing nothing useful. This is the single most important warning in this section: *on imbalanced data, high accuracy can be meaningless*.

0.2.2 Precision: When the Model Says “Positive”, How Often Is It Right?

Definition: Precision

Of all the examples the model *predicted* positive, the fraction that truly are positive:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

Screening example: $40/(40 + 30) = 0.571$. When our model raises the alarm, it is right only 57% of the time.

Intuition & Analogy

Precision is the “**boy who cried wolf**” metric. A model with low precision raises many false alarms, and people stop trusting its warnings. Prioritise precision when a false positive is expensive or corrosive: flagging a legitimate email as spam (the recipient misses something important), or accusing a genuine transaction of fraud (an angry customer whose card was blocked on holiday).

0.2.3 Recall: Of All the Real Positives, How Many Did We Catch?

Definition: Recall (Sensitivity, True Positive Rate)

Of all the examples that are *actually* positive, the fraction the model found:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

Screening example: $40/(40 + 10) = 0.80$. The model catches 80% of the sick patients — and misses 20%.

Intuition & Analogy

Recall is the **fishing-net** metric: of all the fish in the lake, what fraction did the net capture? Prioritise recall when a miss is catastrophic: missing a cancer case, letting a fraudulent transaction through, failing to detect a safety fault. In such settings we accept more false alarms as the price of catching nearly everything.

0.2.4 The Precision–Recall Tug of War

Precision and recall usually pull in opposite directions, and the classifier’s **decision threshold** is the rope:

- **Raise the threshold** (say from 0.5 to 0.9): the model flags only its most confident cases. Fewer false alarms $\Rightarrow$ precision rises; but more true cases slip below the bar $\Rightarrow$ recall falls.
- **Lower the threshold** (say to 0.2): the model flags anything remotely suspicious. It catches almost every true case $\Rightarrow$ recall rises; but it also drowns in false alarms $\Rightarrow$ precision falls.

Neither extreme is “correct”. The right balance depends on the relative cost of a false alarm versus a miss — a judgement about the real world, not about mathematics.

0.2.5 F1-Score: One Number That Respects Both

Definition: F1-Score

The harmonic mean of precision and recall:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

Why the harmonic mean instead of the ordinary average? Because the harmonic mean is dragged down hard by the weaker of the two numbers. If precision = 1.0 but recall = 0.02, the ordinary average is a flattering 0.51, while the F1 is a truthful 0.039. *You cannot buy a good F1 by excelling at one and ignoring the other.*

Screening example: $F_1 = 2 \times \frac{0.571 \times 0.80}{0.571 + 0.80} = 2 \times \frac{0.457}{1.371} \approx 0.667$.

0.2.6 Specificity: Recall for the Negative Class

Definition: Specificity (True Negative Rate)

Of all the truly negative examples, the fraction correctly identified as negative:

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (5)$$

Screening example: $920/(920 + 30) = 0.968$. The model clears 96.8% of healthy patients correctly.

0.2.7 Complete Worked Example: All Metrics from One Matrix

Worked Example: Full Metric Report for the Disease Screener

With $TP = 40$, $FN = 10$, $FP = 30$, $TN = 920$:

Metric	Computation	Value
Accuracy	$(40 + 920)/1000$	0.960
Precision	$40/(40 + 30)$	0.571
Recall	$40/(40 + 10)$	0.800
Specificity	$920/(920 + 30)$	0.968
F1-score	$2(0.571)(0.800)/(0.571 + 0.800)$	0.667

The story in one paragraph: the model *looks* superb by accuracy (96%), catches a decent share of sick patients (recall 80%), but almost half of its alarms are false (precision 57%), and it still misses 1 in 5 sick patients. Whether that is acceptable depends on the disease: for a deadly, treatable condition, recall must be pushed higher (lower the threshold) even at the cost of more false alarms.

0.2.8 Reading a `classification_report`

scikit-learn's `classification_report` prints precision, recall, F1, and `support` per class, plus macro and weighted averages. Two reading rules matter most:

- `support` is simply how many true examples each class has — it exposes class imbalance at a glance (e.g. 950 Healthy vs 50 Sick).
- The `weighted avg` F1 is flattering when a huge majority class dominates it; the `macro avg` reveals weaker performance on the rare class. On imbalanced data, read the *per-class* rows and the *macro* average.

Hands-On Python Notebook

The full Python implementation for this topic lives in the companion notebook `04_Classification_Metrics.ipynb`. It covers computing and displaying the confusion matrix, accuracy, precision, recall, F1, `classification_report`, and the ROC curve with AUC — first on the disease-screening example, then end-to-end on a real classifier trained on the Titanic dataset (`titanic_train.csv`).

0.2.9 Which Metric Should You Optimise?

Table 2: Choosing a metric by the cost structure of the problem.

Situation	Optimise	Reason
Balanced classes, equal error costs	Accuracy	Simple and adequate
False alarms are costly (spam filter)	Precision	Do not block legitimate email
Misses are costly (cancer screening)	Recall	A missed case can be fatal
Imbalanced data, both errors matter	F1 (or PR curve)	Balances the two error types
Comparing models before choosing a threshold	AUC	Threshold-independent ranking quality

Common Mistakes

- **Reporting only accuracy on imbalanced data** — the “predict everyone healthy” trap.
- **Confusing precision with recall.** Anchor: precision divides by *predicted* positives; recall divides by *actual* positives.
- **Feeding hard 0/1 labels into `roc_auc_score`** instead of probability scores — it runs, but the AUC is misleading.
- **Computing metrics on the training set.** Always evaluate on held-out test data.
- **Tuning the threshold on the test set.** Choose the threshold on validation data, then report untouched test performance once.

Key Points

- Every metric is a different ratio of TP, TN, FP, FN.
- Accuracy misleads on imbalanced data; precision and recall expose the two error types separately.
- Precision and recall trade off through the decision threshold; F1 balances them.
- ROC/AUC evaluates ranking quality across all thresholds.
- The “best” metric is dictated by real-world error costs, not by mathematics.